

README.md

Note: All the utility script and metrics have been archived into their respective folders. If you want to use something extracted from the folder first.

Convolution Benchmarking (1D & 2D)

This repository contains implementations of **1D and 2D convolution kernels in C**, designed to study:

- Sequential execution (`conv1d` , `conv2d`)
- Optimised builds with `O3` (`conv1d` , `conv2d`)
- Parallel execution with OpenMP (`conv1d_omp` , `conv2d_omp`)
- Optimised OpenMP builds (`conv1d_omp` , `conv2d_omp` with `O3`)

The project is focused on understanding **performance scaling** under:

- Different compiler optimisations (`O0` vs `O3`)
- Parallelisation strategies (OpenMP, varying threads & schedules)
- Input size growth ($N \times K$ in 1D, $H \times W \times kH \times kW$ in 2D)

We provide:

- A `Makefile` to build all executables with consistent flags
- SLURM job scripts for **UWA's Kaya cluster** (`cits3402` partition)
- Stress sweep scripts for systematic scaling experiments
- Python/Jupyter notebooks for result analysis and visualisation

Convolution Benchmarks — 1D & 2D (Seq & OpenMP)

This repository contains **sequential** and **OpenMP** implementations of **1-D** and **2-D** convolution/correlation with:

- SAME/FULL output modes
- Multiple padding policies
- Reproducible RNG
- Assignment-style I/O formats
- Per-run **CSV metrics** for analysis
- `O3` builds matching the flags used on Kaya (SLURM cluster)

Binaries produced:

- conv1d — 1-D, sequential (O3)
- conv1d_omp — 1-D, OpenMP (O3)
- conv2d — 2-D, sequential (O3)
- conv2d_omp — 2-D, OpenMP (O3)

1) Build with Make (recommended)

Quick start

```
make          # builds conv1d, conv1d_omp, conv2d, conv2d_omp (all O3)
make clean    # removes built binaries
```

Compiler & OpenMP notes

- The Makefile defaults to CC=cc. To force GCC/Clang:

```
make CC=gcc
```

-
- OpenMP (for _omp binaries) uses -fopenmp.
 - **Linux/GCC:** works out of the box.
 - **macOS/Clang:** install libomp (brew install libomp), then build with:

```
make LIBS="-lomp"
```

Build flags (Kaya-style)

All O3 builds use:

```
-std=c11 -O3 -march=native -funroll-loops -fopt-info-vec
```

OpenMP builds additionally use:

```
-fopenmp
```

2) Run via Make (convenience runners)

The Makefile includes **run targets** with sensible defaults. You can override parameters inline.

Common variables:

- 1D: N (length), K (kernel length), SEED, OUT1D=/dev/null
- 2D: H, W, KH, KW, SEED, OUT2D=/dev/null

- OMP: THREADS, SCHED (static|dynamic|guided|auto), CHUNK (optional)

Run all four

```
make run
```

Run each individually

```
# 1D sequential
```

```
make run1d N=1000000 K=10001 SEED=42
```

```
# 1D OpenMP
```

```
make run1d_omp N=1000000 K=10001 THREADS=16 SCHED=guided CHUNK=2048 SEED=42
```

```
# 2D sequential
```

```
make run2d H=2048 W=2048 KH=7 KW=7 SEED=42
```

```
# 2D OpenMP
```

```
make run2d_omp H=1280 W=1920 KH=70 KW=70 THREADS=32 SCHED=auto SEED=42
```

All make runners write outputs to /dev/null by default (to avoid large files). Change with OUT1D=... / OUT2D=....

3) Direct CLI Usage (run binaries yourself)

Shared ideas across tools

- **RNG:** -s, --seed <int> sets deterministic seed; otherwise defaults to current time.
- **Modes:** -m, --mode same|full
- **Padding (SAME only):** -p, --padding zero|none|const; if const, supply -c, --cval <float>
- **I/O:**
 - **1D Input from files:** -f <f.txt>, -g <g.txt>; or generate with sizes (-L, -kL).
 - **2D Input from files:** -f <img.txt>, -g <ker.txt>; or generate with sizes (-H -W -kH -kW).
 - **Output (required):** -o <out.txt>

3.1) 1-D Sequential —

conv1d

Synopsis

```
./conv1d \
[-f f.txt | -L N | --len N] \
[-g g.txt | -kL K | --klen K] \
-o out.txt \
[-s seed|--seed seed] \
```

```
[-m same|full|--mode same|full] \  
[-p zero|none|const|--padding ...] \  
[-c value|--cval value]
```

Examples

```
# Random inputs, SAME (default), ZERO pad (default)  
./conv1d -L 1000000 -kL 10001 -o y.txt -s 42  
  
# File inputs, FULL mode, CONST padding with cval  
./conv1d -f f.txt -g g.txt -m full -p const -c 0.5 -o y.txt
```

1D I/O formats

- **Input files:** first line is L, followed by L floats (whitespace-separated).
- **Output file:** first line is output length, then floats (3 decimals).

3.2) 1-D OpenMP —

conv1d_omp

Same CLI as conv1d **plus** OpenMP controls:

```
-t, --threads <int>      # default 1 (Make runner sets it for you)  
-S, --schedule <kind>    # static|dynamic|guided|auto  
-C, --chunk <int>        # chunk size (optional)
```

Examples

```
# 8 threads, guided schedule with chunk=1024  
./conv1d_omp -L 1e6 -kL 10001 -o /dev/null -s 42 -t 8 -S guided -C 1024  
  
# Rely on env variables instead:  
export OMP_NUM_THREADS=32  
export OMP_SCHEDULE=static:2048  
./conv1d_omp -L 1e6 -kL 10001 -o /dev/null -s 42
```

3.3) 2-D Sequential —

conv2d

Operation: By default runs **convolution**; --corr switches to correlation. Convolution is implemented as correlation with a pre-flipped kernel (flip occurs **outside** the timed region).

Synopsis

```
./conv2d \  
[--conv|--corr] \  
[-f img.txt | -H H --rows H -W W --cols W] \  
[...]
```

```
[-g ker.txt | -kH KH --krows KH -kW KW --kcols KW] \
-o out.txt \
[-s seed|--seed seed] \
[-m same|full|--mode ...] \
[-p zero|none|const|--padding ...] \
[-c value|--cval value]
```

Examples

```
# Convolution (default), random 1024×1024 image + 5×5 kernel
./conv2d -H 1024 -W 1024 -kH 5 -kW 5 -o Y.txt -s 42
```

```
# Correlation, FULL, file inputs
./conv2d --corr -f img.txt -g ker.txt -m full -o Y.txt
```

2D I/O formats

- **Input files:** first line is H W, then H*W floats in **row-major** (whitespace-separated).
- **Output file:** first line outH outW, then outH lines with outW floats (3 decimals, space-separated).

3.4) 2-D OpenMP —

conv2d_omp

Same CLI as conv2d **plus** OpenMP controls (-t/--threads, -S/--schedule, -C/--chunk), identical to 1D OMP.

Examples

```
# 16 threads, SAME, ZERO pad
./conv2d_omp -H 2048 -W 2048 -kH 7 -kW 7 -o /dev/null -s 42 -t 16 -S static
```

```
# FULL mode, correlation, dynamic schedule
./conv2d_omp --corr -H 2048 -W 2048 -kH 11 -kW 11 -m full -o /dev/null -t 32 -S dynamic -C 64
```

4) What gets timed? What gets logged?

- **Timing:** Only the **core kernel** (correlation kernels) is timed. Pre-processing (e.g., 2D kernel flip for convolution), file I/O, and RNG are **excluded** from timings.
- **Metrics CSV:** Every run writes a one-row CSV under metrics/... with fields like:

1D (

conv1d

,

conv1d_omp

)

```
RunID,N,K,outLen,mode,padding,cval,time_s,gflops,threads,schedule,chunk
```

2D (

conv2d

,

conv2d_omp

)

```
RunID,H,W,KH,KW,outH,outW,op,mode,padding,cval,time
```

- **RunID** is unique per run:
 - SLURM_<JOBID> on clusters
 - LOCAL_YYYYMMDD_HHMMSS_<PID> locally

Use these CSVs for downstream notebooks/plots.

5) SLURM usage (examples)

You have parameterized SLURM scripts for 1D and 2D (seq & OMP). The common pattern is:

```
# Submit one parametric job (example 2D OMP)
sbatch conv2d_omp_param.slurm 12800 19200 70 70 32 static 2048 same zero 42

# Serial sweeps that submit and block until CSV+stderr appear:
./stress_sweep_serial_2d.sh HMIN HMAX H_STEP WMIN WMAX W_STEP KHMN KHMN KH_STEP KWMI
N KWMAX KW_STEP [POST_WAIT] [RETRIES] [SEED]
```

Cancel jobs

```
squeue -u $USER      # list your jobs
scancel <JOBID>      # cancel one
scancel -u $USER     # cancel all of yours (careful!)
```

Job states

PD (Pending) with reason (Resources) ⇒ waiting for CPUs/partition limits; R ⇒ running.

6) Reproducibility & Validation

- **RNG:** Always pass -s 42 (or your chosen seed) to make RNG-generated inputs reproducible across runs.
- **Comparison tools:**
 - **1D:** simple Python that compares header length and values within tolerance.

- **2D:** compare_2d.py: loads two H W matrices, reports max/mean abs error, RMSE, and count outside (atol, rtol).

Example 2D compare:

```
./conv2d -H 256 -W 256 -kH 5 -kW 5 -o out_seq.txt -s 42
./conv2d_omp -H 256 -W 256 -kH 5 -kW 5 -o out_omp.txt -s 42
python3 compare_2d.py -A out_seq.txt -B out_omp.txt --atol 1e-6 --rtol 1e-6
```

7) Performance Tips

- **Thread binding (OMP):**
OMP_PROC_BIND=spread and OMP_PLACES=cores often stabilise scaling on multi-socket systems.
- **Schedules:**
static is a great default for regular workloads; try guided/dynamic for very uneven shapes.
- **Chunking:**
For large outputs, a chunk like 1-4× LLC-friendly tile rows can help; otherwise rely on your binary's --chunk or OMP_SCHEDULE=static:<chunk>.
- **Cache-friendliness (2D):**
Output-driven loops with tight index bounds (as implemented) reduce branch mispredictions and help cache reuse on F and G.

8) Troubleshooting

- **OpenMP not enabled (macOS):**
Install libomp and build with make LIBS="-lomp", run _omp binaries only when OpenMP is available.
- **"file format" errors:**
Check the input file headers:
 - 1D: first line must be a single integer L, then L floats.
 - 2D: first line must be H W, then H*W floats in row-major order.
- **No CSV emitted:**
Ensure metrics/... dirs are writable and not redirected to /dev/null. On SLURM, you'll see metrics/o0/metrics_SLURM_<JOBID>.csv (or similar) after the job exits.

9) Examples at a Glance

1D Seq (random):

```
./conv1d -L 107040 -kL 8192 -o y.txt -s 42 -m same -p zero
```

1D OMP (random):

```
export OMP_NUM_THREADS=16
export OMP_SCHEDULE=static:1024
./conv1d_omp -L 1000000 -kL 10001 -o /dev/null -s 42
```

2D Seq (files):

```
./conv2d -f img.txt -g ker.txt -o out.txt -m full
```

2D OMP (random, correlation, FULL)

```
./conv2d_omp --corr -H 2048 -W 2048 -kH 11 -kW 11 -m full \  
-o /dev/null -s 42 -t 32 --schedule guided --chunk 128
```